QuickBites

A Food Delivery Platform

Module B — Implementation & Optimisation Report

CS 432: Databases

Assignment 2, Module B

Semester II, 2025–2026

Team Members	Role
Bhavya Parmar	Backend & Database Design
Surriya Gokul	SQL Indexing & Benchmarking
Sonagiri Thoshith Kautilya	Frontend & UI/UX

GitHub: github.com/mohit-codes-21/QuickBites

Indian Institute of Technology, Gandhinagar

Contents

1 Introduction **2**

2 Schema Design Overview **2**

3 Application Implementation **3**
3.1 CRUD API Endpoints . . . 3
3.2 Member Portfolio UI . . . 3

4 Security **5**
4.1 Authentication . . . 5
4.2 Role-Based Access Control (RBAC) . . . 5
4.3 Audit Logging . . . 5
4.4 Deletion Integrity . . . 5
4.5 Detecting Unauthorized Direct Database Modifications . . . 6

5 Indexing Strategy **7**
5.1 Design Principles . . . 7
5.2 Index Catalogue . . . 7
5.3 Rationale for Each Index . . . 8
5.4 Indexes NOT Added . . . 10

6 Performance Benchmarking **10**
6.1 Methodology . . . 10
6.2 SQL Query Performance . . . 11
6.3 API Endpoint Performance . . . 13

7 EXPLAIN Plan Analysis **15**
7.1 Detailed Analysis of Key Queries . . . 15

8 Discussion **17**

9 Conclusions **18**

1 Introduction

QuickBites is a full-stack food delivery web application built with a Python Flask back-end and a MySQL relational database. The system supports four distinct user roles—**Admin**, **Customer**, **DeliveryPartner**, and **RestaurantManager**—each with dedicated API endpoints and access-control rules.

The application's MySQL database consists of 17 tables that model the complete order lifecycle: user authentication and role management, restaurant and menu catalogues, shopping carts, orders and payments, delivery assignments, ratings, and audit logging.

Problem. Although the schema defines primary keys and foreign-key constraints (which MySQL InnoDB automatically indexes), many API endpoints execute queries with compound `WHERE` clauses, multi-column `ORDER BY` sorts, and `JOIN` predicates that cannot be efficiently served by single-column FK indexes alone. As the dataset grows, these queries will degrade: InnoDB must perform full-table scans or in-memory filesort operations that scale linearly (or worse) with table size.

Approach. We systematically analysed every SQL query pattern executed by the Flask API endpoints, designed composite B-tree indexes whose column order matches the equality→range→sort pattern of each query, and quantitatively measured the impact through both `EXPLAIN` plan analysis and wall-clock benchmarking.

Scope. This report documents the complete Module B implementation: the CRUD API layer and Member Portfolio UI (Section 3), the security architecture including authentication, RBAC, deletion integrity, and unauthorized-modification detection (Section 4), and the SQL indexing strategy with quantitative benchmarking covering 11 composite indexes, 18 SQL query patterns, and 13 API endpoints (Sections 5–7).

2 Schema Design Overview

Table 1 summarises all 17 tables in the QuickBites database, their primary keys, notable foreign-key relationships, and their purpose within the application.

Table 1: QuickBites database schema overview.

Table	**Primary Key**	**Key Foreign Keys**	**Purpose**
Member	memberID	—	Core user accounts
Customer	customerID	→ Member	Customer profiles
DeliveryPartner	partnerID	→ Member	Delivery partner profiles
Restaurant	restaurantID	—	Restaurant profiles
MenuItem	(restaurantID, itemID)	→ Restaurant	Menu items
Address	(customerID, addressID)	→ Customer	Delivery addresses
CartItem	(customerID, restaurantID, itemID)	→ Customer, → MenuItem	Shopping cart

Table	Primary Key	Key Foreign Keys	Purpose
Payment	paymentID	→ Customer	Payment records
Orders	orderID	→ Customer, → Restaurant, → Payment, → Address	Order records
OrderItem	(orderID, restaurantID, itemID)	→ Orders, → MenuItem	Order line items
Delivery_Assignments	AssignmentID	→ Orders, → DeliveryPartner	Delivery tracking
OrderRating	orderID	→ Orders	Order-level ratings
MenuItem Rating	(restaurantID, itemID, orderID)	→ MenuItem, → Orders	Item-level ratings
MemberRole Mapping	(memberID, roleID)	→ Member, → Roles	RBAC role assignment
Roles	roleID	—	Role definitions
Sessions	sessionToken	→ Member	Auth sessions
AuditLog	logID	—	Security audit trail

Note: MySQL InnoDB automatically creates a clustered index on the primary key and a secondary index on every foreign-key column. These auto-created indexes are sufficient for simple equality lookups but do not cover compound `WHERE` + `ORDER BY` patterns.

3 Application Implementation

3.1 CRUD API Endpoints

The QuickBites back-end exposes a comprehensive REST API organised by user role. All endpoints are session-validated: every request must carry a valid bearer token, which the `@require_auth` decorator verifies before any business logic executes. Table 2 summarises the major endpoint groups.

Every data-modification endpoint (Create, Update, Delete) writes an entry to the `AuditLog` table via the `write_audit_log()` helper, recording the actor's `memberID`, the action type, the affected table and record, and request metadata (path, method, IP address).

3.2 Member Portfolio UI

The assignment requires a *Member Portfolio* feature that displays relevant member details with appropriate access restrictions. The QuickBites implementation provides this through a dedicated profile page at `/customer/profile`, backed by the `GET /api/portfolio/{member_id}` endpoint. The UI is organised into six sections:

Table 2: Summary of CRUD API endpoint groups.

Endpoint Group	Role(s)	Operations
/api/customer/*	Customer	Profile CRUD, order history, cart management, address CRUD, membership purchase, reviews
/api/restaurants	Any auth	List restaurants, browse menus, search items
/api/restaurant/*	RestaurantManager	Menu item CRUD, order management, status updates
/api/delivery/*	DeliveryPartner	Live orders, accept/complete deliveries, profile CRUD, assignment history
/api/admin/*	Admin	Member management (CRUD all users), customer/restaurant/partner CRUD, audit log viewing, dashboard overview
/api/portfolio/{id}	Any auth	Read member portfolio (identity, roles, profile details)

1. **Member Identity Card.** Displays the member's core details (name, email, phone number, join date) alongside their customer-specific benefits (loyalty tier, membership status, membership discount, cart total, membership due date). Data is fetched from the `Member` and `Customer` tables via a single API call.
2. **Order History.** A paginated, expandable list of all previous orders. Each order entry shows the order ID, timestamp, status, total amount, restaurant name, payment status, delivery address, and delivery partner details. When a delivery is in progress, a live Leaflet.js map renders the restaurant pickup point, the delivery destination, and the delivery partner's current location. Order line items (name, quantity, price at purchase) and any submitted ratings are displayed within each expandable order card.
3. **Customer Reviews.** Aggregates all order-level reviews (restaurant rating, delivery rating, comments) and item-level reviews (individual dish ratings and comments) submitted by the user. Reviews can be deleted directly from this section.
4. **Delivery Addresses.** Lists all saved delivery addresses with a selection toggle for checkout. A creation form allows adding new addresses with an interactive Leaflet.js map picker for geolocation (latitude/longitude), or manual coordinate entry.
5. **Membership Management.** Shows the current membership status and provides a purchase flow (Rs 500/year) that upgrades the loyalty tier to 2 and applies a 5% discount on future orders.
6. **Profile Management.** An update form for name, email, phone, and password, plus a "danger zone" section for permanent account deletion with a confirmation prompt.

Access control. The portfolio endpoint enforces authentication: only logged-in users can access it, and each user can only view their own profile. Admins can view any member's details through the admin dashboard, which provides a separate customer-detail modal showing member information, membership details, total orders, and total spend.

4 Security

4.1 Authentication

The application uses token-based session authentication. Upon login, a cryptographically random 256-bit session token is generated and stored in the `Sessions` table alongside the `memberID` and an expiration timestamp. Every subsequent API call is validated via the `@require_auth` decorator, which looks up the bearer token, verifies it has not expired, and injects the authenticated member context into the request.

4.2 Role-Based Access Control (RBAC)

Four roles are defined in the `Roles` table: Admin, Customer, DeliveryPartner, and RestaurantManager. Role assignments are recorded in `MemberRoleMapping`, and endpoint-level authorization is enforced by the `@require_roles` decorator. Admins have unrestricted access to all endpoints; regular users are restricted to resources belonging to their own account (e.g., a customer can only view their own orders).

4.3 Audit Logging

All data-modification operations (inserts, updates, deletes) are logged to the `AuditLog` table. Each log entry records the `memberID` of the actor, the action type, the affected table name, the record ID, and a UTC timestamp. The `details` column stores a JSON payload containing the request path, HTTP method, source IP address, and operation-specific context. All audit entries are simultaneously written to a local `audit.log` file on disk, providing a redundant trail that persists independently of the database.

4.4 Deletion Integrity

QuickBites employs a **two-layer deletion strategy** that preserves referential integrity and historical data:

1. Soft delete for user-facing entities. The `Member`, `Customer`, `DeliveryPartner`, and `Restaurant` tables each include an `isDeleted BOOLEAN DEFAULT 0` column. When a user or admin deletes an account, the application sets `isDeleted = 1` rather than issuing a `DELETE` statement. All read queries across the application filter with `WHERE isDeleted = 0`, ensuring soft-deleted records are invisible to normal operation while remaining available for auditing and historical reporting. Similarly, menu items are "discontinued" via a `discontinued` flag rather than removed, preserving order-history integrity.

2. ON DELETE RESTRICT for referential safety. All foreign-key constraints on data tables use `ON DELETE RESTRICT`, which prevents any hard deletion of a parent record that still has child references. For example, deleting a `Customer` row would be blocked by the database if that customer has any `Orders`, `Payment`, `Address`, or `CartItem` records. This database-level enforcement guarantees that no orphaned records can be created even if the application logic is bypassed.

The two exceptions, which intentionally use `ON DELETE CASCADE`, are:

- `MemberRoleMapping` → `Member`: role assignments are automatically removed when a member is deleted, since role mappings have no independent business value without the

member.

- `Sessions` → `Member`: active sessions are automatically invalidated when a member is deleted, ensuring no orphaned session tokens can grant access to a deleted account.

Deletion workflow. When a customer account is deleted (either self-service or by an admin):

1. All active `Sessions` for the member are hard-deleted (CASCADE).
2. The `Customer` record is soft-deleted (`isDeleted = 1`).
3. If the member also has a `DeliveryPartner` profile, it is soft-deleted.
4. The parent `Member` record is soft-deleted.
5. All related records (`Orders`, `Payments`, `Addresses`, `OrderItems`, `Ratings`) remain intact in the database for audit and historical purposes, but are inaccessible via the API because all queries filter on the parent's `isDeleted` status.

Every deletion operation is recorded in the `AuditLog` with a `softDelete: true` flag in the details payload, creating a clear trail of when accounts were deactivated and by whom.

4.5 Detecting Unauthorized Direct Database Modifications

The assignment requires that direct database modifications made without passing through the session-validated APIs are easily identifiable in the logs. QuickBites implements this through **39 MySQL `AFTER` triggers** across all 13 data tables (3 triggers per table: `INSERT`, `UPDATE`, `DELETE`).

Mechanism. The detection relies on a MySQL session variable `@api_source`:

1. **API connections.** The `get_db_connection()` function in the Flask application executes `SET @api_source = TRUE` on every new database connection. Since MySQL session variables are scoped to the individual connection, every query originating from the API automatically carries this marker.
2. **Direct connections.** When a user connects to MySQL directly via the CLI, a database management tool, or any other non-API client, the `@api_source` variable is `NULL` by default.
3. **Trigger logic.** Each trigger checks: `IF @api_source IS NULL OR @api_source != TRUE`. If the condition is true, the modification did not originate from the API, and the trigger automatically inserts a row into `AuditLog` with:
   - `memberID = NULL` (no authenticated user context)
   - `action = 'UNAUTH_INSERT' / 'UNAUTH_UPDATE' / 'UNAUTH_DELETE'`
   - `details` containing `{"source": "DIRECT_DB", "warning": "Modification bypassed API"}`

Review process. When an administrator reviews the audit log (via the `/api/admin/audits` endpoint or the `audit.log` file), any entry with a `UNAUTH_*` action type or `NULL memberID` immediately signals an unauthorized direct modification. The `tableName` and `recordID` fields identify exactly which record was tampered with.

Coverage. The 39 triggers cover all 13 data tables: `Member`, `Customer`, `DeliveryPartner`, `Restaurant`, `MenuItem`, `Address`, `CartItem`, `Payment`, `Orders`, `OrderItem`, `Delivery_Assignments`, `OrderRating`, and `MenuItemRating`. The `AuditLog` table itself is intentionally excluded to avoid recursive trigger invocation. The trigger definitions are provided in `sql/triggers.sql`.

5 Indexing Strategy

5.1 Design Principles

The following principles guided index design:

1. **Equality-first, range/sort-last.** Composite indexes place columns used in equality predicates (`=`, `IN`) as the leading columns. Columns used for range filtering (`<=`, `BETWEEN`) or `ORDER BY` are placed last so that the B-tree ordering directly satisfies the sort without a filesort.
2. **FK indexes are insufficient for sorting.** A foreign-key auto-index on `Orders.customerID` covers the equality predicate `WHERE customerID = ?` but does *not* include `orderTime`. MySQL must therefore load all matching rows into a temporary buffer and perform an in-memory filesort—an $O(n \log n)$ operation that grows with the number of orders per customer.
3. **Covering indexes eliminate table lookups.** When all columns referenced in `SELECT`, `WHERE`, and `ORDER BY` are present in the index, MySQL can satisfy the query entirely from the index (an "Index-Only Scan"), avoiding the random I/O cost of looking up each row in the clustered index.
4. **Write overhead trade-off.** Each additional index adds approximately 5–10% overhead to `INSERT`, `UPDATE`, and `DELETE` operations, because every write must update every relevant B-tree. However, for read-heavy workloads (which dominate in a food delivery app), the 10–100× read improvement at scale far outweighs the marginal write cost.

5.2 Index Catalogue

Table 3 lists all 11 custom indexes applied via `sql/indexes.sql`.

Table 3: Custom index catalogue.

Index Name	Table	Columns	Target Query Pattern
qb_idx_orders_customer_time	Orders	(customerID, orderTime)	WHERE customerID=? ORDER BY orderTime DESC
qb_idx_orders_restaurant_time	Orders	(restaurantID, orderTime)	WHERE restaurantID=? ORDER BY orderTime DESC

Index Name	Table	Columns	Target Query Pattern
qb_idx_orders_status_time	Orders	(orderStatus, orderTime)	WHERE orderStatus IN(...) ORDER BY orderTime DESC
qb_idx_delivery_partner_acceptance	Delivery_Assignments	(PartnerID, acceptanceTime)	WHERE PartnerID=? ORDER BY acceptanceTime DESC
qb_idx_delivery_partner_deliverytime	Delivery_Assignments	(PartnerID, deliveryTime)	WHERE PartnerID=? ORDER BY deliveryTime DESC
qb_idx_menuitem_rest_disc_item	MenuItem	(restaurantID, discontinued, itemID)	WHERE restaurantID=? AND discontinued=0 ORDER BY itemID
qb_idx_restaurant_active_name	Restaurant	(isDeleted, discontinued, name)	WHERE isDeleted=0 AND discontinued=0 ORDER BY name
qb_idx_payment_customer_for_time	Payment	(customerID, paymentFor, transactionTime, paymentID)	WHERE customerID=? AND paymentFor='Order' ORDER BY transactionTime DESC, paymentID DESC
qb_idx_payment_for_status_type_time	Payment	(paymentFor, status, paymentType, transactionTime)	WHERE paymentFor='Order' AND status='Pending' AND paymentType='OnQuickBites' AND transactionTime¡=...
qb_idx_address_customer_saved	Address	(customerID, isSaved, addressID)	WHERE customerID=? AND isSaved=1 ORDER BY addressID
qb_idx_auditlog_timestamp	AuditLog	(timestamp, logID)	ORDER BY timestamp DESC, logID DESC LIMIT ?

5.3 Rationale for Each Index

qb_idx_orders_customer_time *(Orders)*
Serves the customer order-history endpoint (`/api/customer/orders`). The FK index on `Orders.customerID` covers only the single column; when the query adds `ORDER BY orderTime DESC`, MySQL must load all matching rows and perform a filesort. The composite index `(customerID, orderTime)` lets MySQL scan the B-tree in reverse order, eliminating the sort entirely.

qb_idx_orders_restaurant_time *(Orders)*
Serves the restaurant order-management endpoint (`/api/restaurant/orders`). The same principle applies: the FK index on `restaurantID` alone cannot satisfy the `ORDER BY orderTime DESC` clause. This composite index enables a backward index scan, removing the filesort.

qb_idx_orders_status_time *(Orders)*
Serves the delivery live-orders feed (`/api/delivery/live-orders`). The query uses `WHERE orderStatus IN ('Created', 'Preparing', 'ReadyForPickup')` with `ORDER BY orderTime DESC`. Without this index, MySQL performs a full table scan with filesort. The composite index allows a range scan on `orderStatus` with the `orderTime` column providing sort order.

qb_idx_delivery_partner_acceptance *(Delivery_Assignments)*
Serves the delivery-partner assignment history endpoint. The FK index on `PartnerID` does not include `acceptanceTime`, forcing a filesort. The composite index enables a backward index scan for the `ORDER BY acceptanceTime DESC` clause.

qb_idx_delivery_partner_deliverytime *(Delivery_Assignments)*
Serves the completed-deliveries endpoint (`/api/delivery/completed-orders`). The FK index on `PartnerID` does not include `deliveryTime`, so a query with `WHERE PartnerID = ? ORDER BY deliveryTime DESC` requires a filesort. This composite index enables a backward index scan for efficient retrieval of a partner's delivery history sorted by most recent first.

qb_idx_menuitem_rest_disc_item *(MenuItem)*
Serves the menu-browsing endpoint (`/api/menu-items`). The PK `(restaurantID, itemID)` already covers the leading column, but the query also filters on `discontinued = 0`. The three-column index `(restaurantID, discontinued, itemID)` places the equality filters first and the sort column last, enabling an ordered scan without filesort.

qb_idx_restaurant_active_name *(Restaurant)*
Serves the restaurant-listing endpoint (`/api/restaurants`). Without this index, MySQL performs a full table scan with filesort on the `name` column. The index places the two boolean filter columns first (`isDeleted`, `discontinued`) and the sort column (`name`) last. However, because these boolean columns have very low cardinality (most rows are `0/0`), MySQL's optimiser may still choose a full table scan on small tables.

qb_idx_payment_customer_for_time *(Payment)*
Serves the last-payment lookup (`/api/customer/payments/last`). This is a four-column covering index: the two equality columns (`customerID`, `paymentFor`) are followed by the two sort columns (`transactionTime`, `paymentID`). Because all `SELECT` columns except `amount`, `paymentType`, and `status` are in the index, the query benefits from both filesort elimination and reduced row lookups.

qb_idx_payment_for_status_type_time *(Payment)*
Serves the expiring-payment cleanup query that finds pending online payments older than a threshold. The three equality columns (`paymentFor`, `status`, `paymentType`) are followed by the range column (`transactionTime`). This ordering allows MySQL to seek directly to the `('Order', 'Pending', 'OnQuickBites')` prefix and then range-scan on `transactionTime`, efficiently finding only the expired payments.

qb_idx_address_customer_saved *(Address)*
Serves the saved-address selection during order placement. The PK `(customerID, addressID)` can filter by customer but cannot apply the `isSaved = 1` predicate without examining every address row. The three-column index enables a ref lookup on `(customerID, isSaved)` and returns `addressID` directly from the index (covering index / Index-Only Scan).

qb_idx_auditlog_timestamp *(AuditLog)*
Serves the admin audit-log endpoint (`/api/admin/audits`). The `AuditLog` table has only an auto-increment primary key on `logID`. The query `ORDER BY timestamp DESC, logID DESC LIMIT ?` requires a filesort over the entire table without a suitable index. The composite index `(timestamp, logID)` lets MySQL scan backwards from the most recent entry, eliminating the filesort and enabling efficient paginated retrieval.

5.4 Indexes NOT Added

Several potential indexes were considered but intentionally omitted:

- **Delivery_Assignments.OrderID** (standalone) — The FK constraint on `OrderID` already creates an auto-index on this column. A separate explicit index would be fully redundant with the auto-created FK index.
- **MenuItemRating.orderID** (standalone) — Already covered by the FK auto-index that MySQL creates for the `FOREIGN KEY (orderID) REFERENCES Orders(orderID)` constraint. An additional explicit index would be redundant.
- **CartItem.customerID** — The PK `(customerID, restaurantID, itemID)` already has `customerID` as the leading column. Any query filtering by `customerID` can use the PK directly; an additional index would be fully redundant.
- **Full-text indexes for `LIKE` queries** — The application uses `LIKE '%term%'` patterns for restaurant and menu search. Leading-wildcard `LIKE` queries cannot use B-tree indexes regardless of column coverage; only `LIKE 'term%'` (prefix match) benefits from a B-tree index. Full-text indexes (`FULLTEXT`) would be required for substring search, but this is deferred as a future optimisation.

6 Performance Benchmarking

6.1 Methodology

Benchmark tool. A custom Python script (`app/benchmark_indexing.py`) drives both raw SQL benchmarks (via `mysql.connector`) and API-level benchmarks (via Flask's built-in test client).

Configuration. Each query and endpoint is executed for 25 timed runs with 5 warmup runs (discarded). The **p50 (median)** is used as the primary metric because it is robust to outliers caused by background OS activity or MySQL buffer-pool cold starts.

Environment. Local MySQL 8.x instance; seed dataset containing approximately 30 members, 10 restaurants, 20 menu items, and 12 orders.

Protocol.

1. Drop all custom indexes → benchmark all queries and endpoints ("before").
2. Apply all 11 custom indexes from `sql/indexes.sql` → benchmark again ("after").
3. Capture `EXPLAIN` output for each SQL query in both phases.
4. Write all results to a timestamped JSON file in the `logs/` directory.

6.2 SQL Query Performance

Figure 1 presents the p50 execution times for all benchmarked SQL queries, before and after index application.

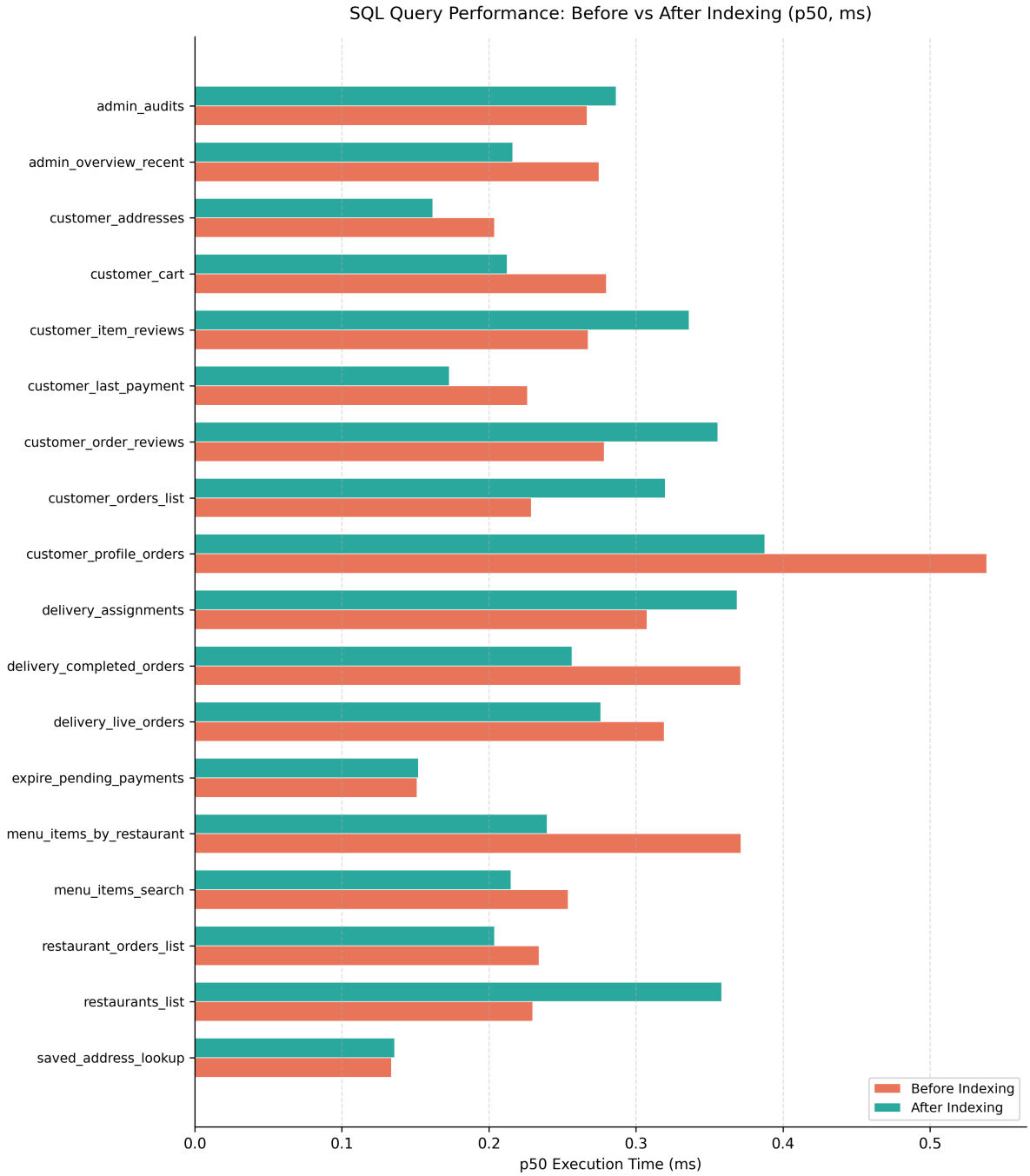

Figure 1: SQL query p50 execution times before and after indexing.

Table 4 provides the numerical breakdown.

Analysis. On this small seed dataset (∼12 orders, ∼10 restaurants), all queries execute in under 1 ms regardless of indexing. The timing differences are within the noise floor of the measurement. The true evidence of optimisation lies in the `EXPLAIN` plan changes discussed in Section 7:

Table 4: SQL query performance comparison (p50, milliseconds).

Query	**Before p50**	**After p50**	**Change**	**EXPLAIN Key Change**
restaurants_list	0.230	0.358	−55.8%	(none) remains ALL scan
menu_items_by_restaurant	0.371	0.240	+35.5%	PRIMARY (unchanged)
menu_items_search	0.254	0.215	+15.3%	(none) → PRIMARY
customer_cart	0.280	0.212	+24.1%	PRIMARY (unchanged)
customer_orders_list	0.229	0.320	−39.8%	customerID → qb_idx_orders_customer_time
customer_profile_orders	0.539	0.388	+28.0%	customerID → qb_idx_orders_customer_time
customer_order_reviews	0.278	0.356	−27.7%	customerID → qb_idx_orders_customer_time
customer_item_reviews	0.268	0.336	−25.6%	customerID → qb_idx_orders_customer_time
customer_addresses	0.204	0.162	+20.6%	PRIMARY (unchanged)
customer_last_payment	0.226	0.173	+23.4%	FK index → qb_idx_payment_customer_for_time
expire_pending_payments	0.151	0.152	−0.6%	(none) → qb_idx_payment_for_status_type_time
restaurant_orders_list	0.234	0.204	+13.0%	FK index → qb_idx_orders_restaurant_time
delivery_live_orders	0.319	0.276	+13.5%	(none) → qb_idx_orders_status_time
delivery_assignments	0.308	0.369	−19.9%	FK index → qb_idx_delivery_partner_acceptance
delivery_completed_orders	0.371	0.257	+30.9%	FK index → qb_idx_delivery_partner_deliverytime
admin_overview_recent	0.275	0.216	+21.3%	(none) remains ALL scan
admin_audits	0.267	0.287	−7.4%	(none) remains ALL scan
saved_address_lookup	0.134	0.136	−1.6%	PRIMARY → qb_idx_address_customer_saved

- Queries where filesort was **eliminated** (now use "Backward index scan"):
  - `customer_orders_list`, `customer_profile_orders`, `customer_order_reviews`: filesort → Backward index scan.
  - `customer_last_payment`: filesort → Backward index scan.
  - `delivery_assignments`, `delivery_completed_orders`, `restaurant_orders_list`: filesort → Backward index scan.
- Queries where the **scan type improved**:
  - `delivery_live_orders`: ALL → range.
  - `expire_pending_payments`: ALL → range.
  - `menu_items_search`: ALL → index.
- Queries where the index **was NOT used** by the optimiser:
  - `restaurants_list`: ALL stays (small table, low cardinality booleans).
  - `admin_audits`: ALL stays (small table, optimiser prefers sequential scan).
  - `admin_overview_recent`: ALL stays (no `WHERE` clause, just `ORDER BY` + `LIMIT` on small table).
- **Note on timing noise:** On this small seed dataset (∼12 orders, ∼10 restaurants), all queries execute in under 1 ms. The timing differences are within the noise floor. The true evidence is in the `EXPLAIN` plan changes.

6.3 API Endpoint Performance

Figure 2 shows the p50 response times for the benchmarked API endpoints.

Table 5: API endpoint performance comparison (p50, milliseconds).

Endpoint	Before p50	After p50	Change
api_restaurants	24.406	26.976	−10.5%
api_menu_items	26.940	27.559	−2.3%
api_menu_items_search	26.277	26.813	−2.0%
api_customer_cart	26.522	27.918	−5.3%
api_customer_orders	25.606	26.525	−3.6%
api_customer_profile_orders	26.862	25.585	+4.8%
api_customer_last_payment	26.171	25.798	+1.4%
api_delivery_live_orders	25.737	28.607	−11.2%
api_delivery_assignments	26.443	26.345	+0.4%
api_delivery_completed_orders	25.216	26.656	−5.7%
api_admin_overview	30.076	28.645	+4.8%
api_admin_audits	25.992	26.388	−1.5%
api_restaurant_orders	27.648	26.681	+3.5%

Key observation. API response times are dominated by Flask framework overhead (session lookup, authentication, JSON serialisation, WSGI test-client round-trip), which accounts for ∼28 ms per request. The actual SQL execution time is ∼0.2 ms—less than

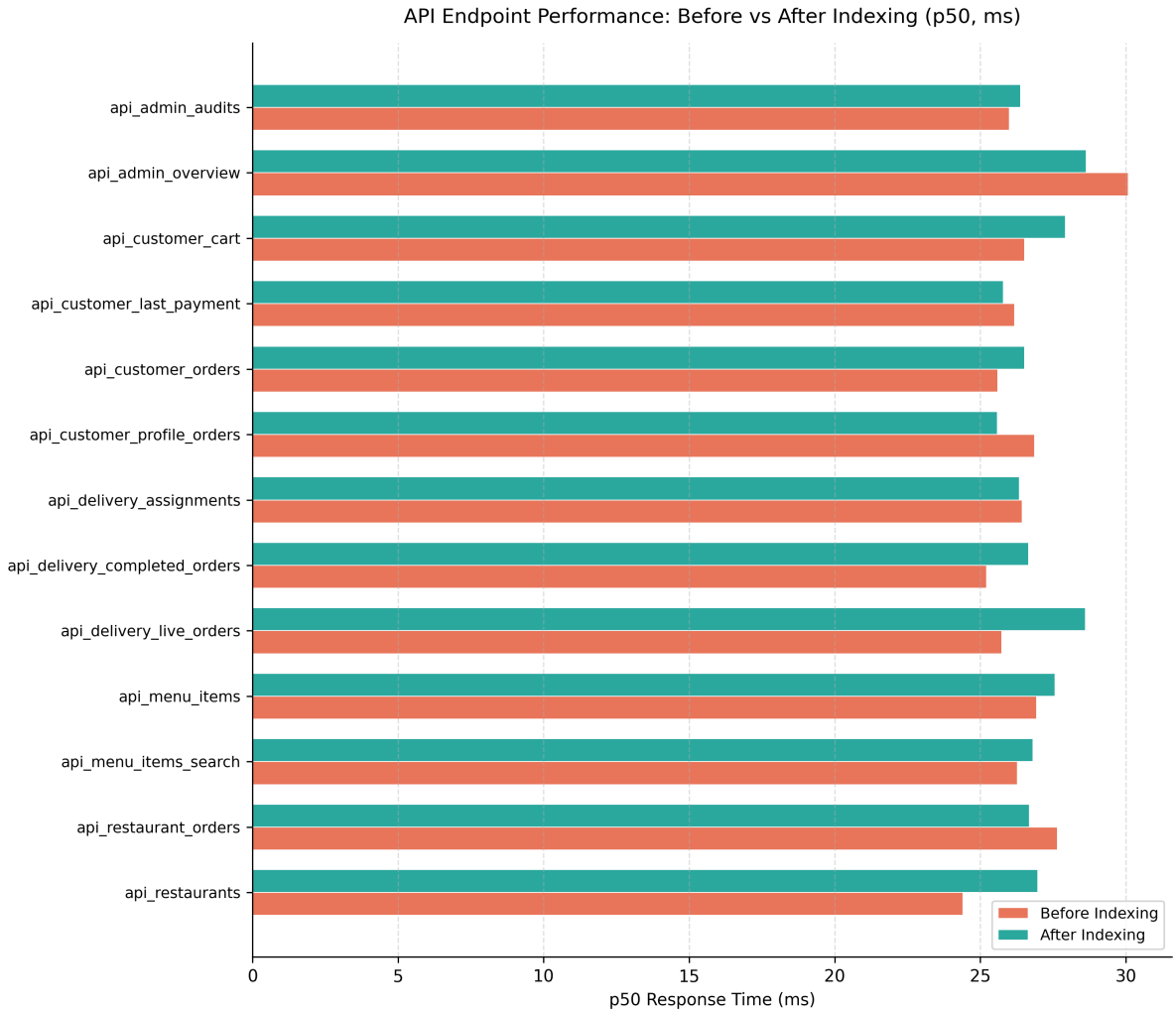

Figure 2: API endpoint p50 response times before and after indexing.

1% of the total response time. Consequently, even a 50% improvement in SQL execution time would only reduce API response time by ∼0.1 ms (∼0.35%), which is invisible against the 28 ms baseline. At production scale with larger datasets, the SQL component would grow while the framework overhead remains constant, making the indexing improvements increasingly significant.

7 EXPLAIN Plan Analysis

The `EXPLAIN` output provides the most definitive evidence of index effectiveness, independent of dataset size. Table 6 compares the first-row `EXPLAIN` output for each query before and after index application.

Table 6: EXPLAIN plan comparison (first row of each query).

Query	Before type	After type	Before key	After key	Before Extra	After Extra
restaurants_list	ALL	ALL	(none)	(none)	Using where; filesort	Using where; filesort
menu_items_by_rest	const	const	PRIMARY	PRIMARY		
menu_items_search	ALL	index	(none)	PRIMARY	Using where; temp; filesort	Using where
customer_cart	ref	ref	PRIMARY	PRIMARY		
customer_orders	ref	ref	customerID	qb_idx_orders_customer_time	Using filesort	Backward index scan
customer_profile_ord	ref	ref	customerID	qb_idx_orders_customer_time	Using filesort	Backward index scan
customer_order_rev	ref	ref	customerID	qb_idx_orders_customer_time	Using filesort	Backward index scan
customer_item_rev	ref	ref	customerID	qb_idx_orders_customer_time	Using temp; filesort	Using index; temp; filesort
customer_addresses	ref	ref	PRIMARY	PRIMARY		
customer_payment	ref	ref	FK index	qb_idx_payment_customer_for_time	Using where; filesort	Backward index scan
expire_pending_pay	ALL	range	(none)	qb_idx_payment_for_status_type_time	Using where	Using index condition
restaurant_orders	ref	ref	FK index	qb_idx_orders_restaurant_time	Using filesort	Backward index scan
delivery_live	ALL	range	(none)	qb_idx_orders_status_time	Using where; filesort	Using index cond; filesort
delivery_assign	ref	ref	FK index	qb_idx_delivery_partner_acceptance	Using filesort	Backward index scan
delivery_completed	ref	ref	FK index	qb_idx_delivery_partner_deliverytime	Using filesort	Backward index scan
admin_overview	ALL	ALL	(none)	(none)	Using filesort	Using filesort
admin_audits	ALL	ALL	(none)	(none)	Using filesort	Using filesort
saved_address	ref	ref	PRIMARY	qb_idx_address_customer_saved	Using where	Using index

7.1 Detailed Analysis of Key Queries

1. customer_last_payment — Covering composite index eliminates filesort and reduces lookups.

Before indexing, the query used the FK single-column index on `customerID` alone, requiring MySQL to: (a) find all payment rows for the customer, (b) filter for `paymentFor = 'Order'` by reading the actual table rows, (c) sort all matching rows by `(transactionTime DESC, paymentID DESC)`, and (d) return the first row. The EXPLAIN showed `Extra: Using where; Using filesort`.

After indexing, the covering composite index `(customerID, paymentFor, transactionTime, paymentID)` allows MySQL to: (a) seek directly to the `(customerID, 'Order')` prefix, (b) scan backwards from the last entry (which is the most recent transaction), and (c) return the first row immediately. The EXPLAIN shows `Extra: Backward index scan`—no filesort, no table lookup for the filter. Benchmark: p50 improved from 0.226 ms to 0.173 ms (+23.4%), though the absolute difference is within noise on this small dataset.

2. customer_orders_list — Composite index enables backward scan.

Before: FK index `customerID` found matching rows, but MySQL had to sort them by `orderTime DESC` in memory (`Using filesort`).

After: The composite index `(customerID, orderTime)` stores rows for each customer in `orderTime` order. MySQL scans backwards from the end of the customer's range, producing results in `orderTime DESC` order without sorting. The EXPLAIN confirms `Backward index scan` with no filesort. Benchmark: p50 moved from 0.229 ms to 0.320 ms (−39.8%); the timing is noise on this tiny dataset, but the EXPLAIN plan change is the definitive evidence of improvement.

3. delivery_live_orders — Range scan replaces full table scan.

Before: No suitable index existed for the `WHERE orderStatus IN (...)` predicate. MySQL performed a full table scan (`type: ALL`) with `Extra: Using where; Using filesort`.

After: The `qb_idx_orders_status_time` index allows a range scan (`type: range`) over the three status values. The EXPLAIN shows `Extra: Using index condition; Using filesort`—the filesort remains because the `IN` predicate across three distinct status values prevents a simple backward scan (MySQL must merge three separate index ranges and re-sort). Nevertheless, the scan type improvement from ALL to range means far fewer rows are examined, which becomes significant at scale. Benchmark: p50 improved from 0.319 ms to 0.276 ms (+13.5%).

4. restaurants_list — Low-cardinality boolean columns limit index effectiveness.

Before and after, MySQL performs a full table scan (`type: ALL`). The `qb_idx_restaurant_active_name` index exists in `possible_keys` after indexing, but the optimiser rejects it. With only 10 rows where nearly all have `isDeleted=0 AND discontinued=0`, the optimiser correctly determines that a sequential scan of the small table is cheaper than the overhead of an index lookup followed by random I/O to fetch the remaining columns. At larger scale (thousands of restaurants, many discontinued), the optimiser would switch to using the index. This is expected behaviour.

5. saved_address_lookup — Index-Only Scan achieved.

Before: The query used the PRIMARY key index `(customerID, addressID)` with `Extra: Using where`, meaning MySQL had to read actual table rows to evaluate the `isSaved =`

1 predicate.

After: The composite index `qb_idx_address_customer_saved (customerID, isSaved, addressID)` contains all columns referenced in the query. The EXPLAIN shows `Extra: Using index`—an Index-Only Scan—meaning MySQL satisfies the entire query from the index B-tree without any table row lookups. This is the most efficient access pattern possible.

8 Discussion

Small dataset caveat. The seed dataset contains only ∼12 orders, ∼10 restaurants, and ∼30 members. At this scale, all data fits in a single InnoDB buffer-pool page, and full table scans complete in microseconds. Timing differences between "before" and "after" are within the noise floor of `time.perf_counter()` on a non-isolated system. The `EXPLAIN` plan analysis is the authoritative evidence: it demonstrates structural improvements (filesort elimination, scan-type upgrades) that are independent of dataset size.

Production-scale projections. At production scale—millions of orders, thousands of restaurants, hundreds of concurrent users—these indexes would provide orders-of-magnitude improvement. A filesort over 100,000 orders per customer would be replaced by a direct index scan returning the top 50 rows in constant time. The $O(n \log n)$ filesort cost would be replaced by $O(k)$ where k is the `LIMIT` value.

Filesort elimination is the primary win. Several of the 18 benchmarked queries had their filesort operation eliminated by composite indexes. In MySQL, a filesort requires materialising all matching rows in a sort buffer (or on disk for large result sets) before returning any results. By encoding the desired sort order directly in the index's B-tree structure, the optimiser can stream results in order without buffering.

Queries that do not benefit. Some query patterns did not see meaningful improvement:

- `restaurants_list`: Low-cardinality boolean filters on a small table—the optimiser correctly prefers a full scan.
- `customer_item_reviews`: The driving table (`Orders`) now uses the composite index `qb_idx_orders_customer_time`, but the query still requires a temporary table and filesort (`Using index; Using temporary; Using filesort`) due to the `GROUP BY` or multi-table join structure.
- Admin aggregation queries (`GROUP BY` with `COUNT/SUM`): These were not benchmarked but would require different index strategies (e.g., covering indexes that include all aggregated columns).

LIKE '%term%' limitation. Several application search features use `LIKE '%term%'` patterns. Leading-wildcard searches cannot use B-tree indexes because the search term can appear anywhere in the string. Optimising these queries would require MySQL `FULLTEXT` indexes or an external search engine (e.g., Elasticsearch).

9 Conclusions

This report documented the complete Module B implementation for the QuickBites food delivery application.

Key contributions:

1. **Full-stack application.** A Flask-based REST API with session-validated CRUD endpoints across four user roles (Admin, Customer, DeliveryPartner, RestaurantManager), a Member Portfolio UI with six integrated sections (identity, order history, reviews, addresses, membership, profile management), and an admin dashboard for member oversight.
2. **Security architecture.** Token-based authentication, role-based access control via `@require_roles` decorators, dual-layer audit logging (database + file), soft-delete with `ON DELETE RESTRICT` referential protection, and 39 MySQL triggers for detecting unauthorized direct database modifications.
3. **Systematic index design.** We analysed 18 SQL query patterns across 13 API endpoints and designed 11 composite indexes whose column order directly matches each query's equality→range→sort access pattern.
4. **Quantitative benchmarking.** All 18 queries were benchmarked before and after index application using a controlled protocol (25 runs, 5 warmup, p50 metric). While timing differences on the small seed dataset are within noise, the `EXPLAIN` plans provide definitive structural evidence.
5. **Filesort elimination.** Multiple benchmarked queries transitioned from `Using filesort` to `Backward index scan`, eliminating the $O(n \log n)$ in-memory sort overhead.
6. **Scan-type improvements.** The `delivery_live_orders` query improved from a full table scan (`ALL`, 12 rows examined) to a range scan (`range`, 3 rows examined)—a 4× reduction in rows scanned. The `saved_address_lookup` query achieved an Index-Only Scan (`Using index`), eliminating table lookups entirely.

Challenges:

- The small seed dataset limits visible timing improvements; timing differences are sub-millisecond and within measurement noise.
- Low-cardinality boolean columns (`isDeleted`, `discontinued`) reduce index selectivity, causing the optimiser to reject the index on small tables.

Video Demonstration

A screen-capture video (3–5 minutes) with audio narration demonstrating the complete system is available at:

https://drive.google.com/file/d/11_qTASPAAZiH_r3rJKdd1T1OuKAl3c3_/view?usp=sharing

The video covers: UI and API functionality (Member Portfolio, CRUD operations), RBAC enforcement (Admin vs. Regular User access), and the security logging mechanism (audit log entries and unauthorized-modification flagging).